



PDF Download
3319499.3328229.pdf
06 April 2026
Total Citations: 14
Total Downloads: 582



Published: 18 June 2019

Citation in BibTeX format

EICS '19: ACM SIGCHI Symposium
on Engineering Interactive Computing
Systems

June 18 - 21, 2019
Valencia, Spain

Conference Sponsors:
SIGCHI

 Latest updates: <https://dl.acm.org/doi/10.1145/3319499.3328229>

POSTER

Component-based development of adaptive user interfaces

ENES YIGITBAS, Paderborn University, Paderborn, Nordrhein-Westfalen, Germany

KLEMENTINA JOSIFOVSKA, Paderborn University, Paderborn, Nordrhein-Westfalen, Germany

IVAN JOVANOVIKJ, Paderborn University, Paderborn, Nordrhein-Westfalen, Germany

FERHAT KALINCI, Paderborn University, Paderborn, Nordrhein-Westfalen, Germany

ANTHONY ANJORIN, Paderborn University, Paderborn, Nordrhein-Westfalen, Germany

GREGOR ENGELS, Paderborn University, Paderborn, Nordrhein-Westfalen, Germany

Open Access Support provided by:

Paderborn University

Component-Based Development of Adaptive User Interfaces

Enes Yigitbas
Paderborn University
Paderborn, Germany
enes@mail.upb.de

Klementina Josifovska
Paderborn University
Paderborn, Germany
klementina.josifovska@upb.de

Ivan Jovanovikj
Paderborn University
Paderborn, Germany
ivan.jovanovikj@upb.de

Ferhat Kalinci
Paderborn University
Paderborn, Germany
fkalinci@mail.upb.de

Anthony Anjorin
Paderborn University
Paderborn, Germany
anthony.anjorin@upb.de

Gregor Engels
Paderborn University
Paderborn, Germany
engels@upb.de

ABSTRACT

Modern User Interfaces (UIs) are increasingly expected to be plastic in the sense that they retain a constant level of usability, even when subjected to context changes at runtime. Adaptive UIs (AUIs) have been promoted as a solution for context variability due to their ability to automatically adapt to the context-of-use at runtime. However, development of AUIs is a complex task as different aspects such as context monitoring and UI adaptation have to be supported. In previous work, model-driven engineering approaches were proposed to support the development of AUIs in a systematic and efficient manner. However, existing model-driven development approaches for AUIs face challenges regarding flexibility, reusability, and compatibility to de facto standard UI frameworks like Angular, which hinder their industry-wide usage and adoption in practice. To address this problem and explore an alternative approach, we propose a component-based development framework for AUIs (CoBAUI). CoBAUI defines a modular framework for supporting the development of AUIs and consists of various components to cover aspects like context monitoring and UI adaptation at widget level. The CoBAUI framework was implemented based on Angular and aims to support the development of AUIs through highly reusable and flexible components. We demonstrate the benefit of our CoBAUI framework based on a case study of an AUI for a library web application.

CCS CONCEPTS

• **Human-centered computing** → **User interface management systems**; **User interface programming**.

KEYWORDS

adaptive user interface; component-based development

ACM Reference Format:

Enes Yigitbas, Klementina Josifovska, Ivan Jovanovikj, Ferhat Kalinci, Anthony Anjorin, and Gregor Engels. 2019. Component-Based Development of Adaptive User Interfaces. In *ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS '19)*, June 18–21, 2019, Valencia, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3319499.3328229>

1 INTRODUCTION

The user interface (UI) acts as a mediator between user and interactive system and it should offer a high level of usability and utility. Considering the variety of mobile devices and dynamically changing context-of-use situations, a ‘one-size-fits-all’ UI cannot guarantee that users’ demands are met in diverse situations. Users of mobile devices have different skills, preferences, and abilities. In order to ensure functionality and usability, applications have to consider contextual information. The term context-of-use reflects the characteristics of *User*, *Platform*, and *Environment* [4]. The ability of an application’s UI to support heterogeneous contexts-of-use while maintaining usability is referred to as plasticity [3].

Adaptive UIs (AUIs) have been promoted as a solution for context variability due to their ability to automatically adapt to the context-of-use at runtime. However, the development of AUIs is a complex task and involves significant effort and complexity. To facilitate this work, model-driven approaches such as *RBUIs* [1] or *AdaptUI* [11] were introduced in recent research to reduce the implementation effort and automatically generate adaptive UIs. Although these approaches support the development of adaptive UIs, they face challenges regarding flexibility, reusability, and standardization with

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

EICS '19, June 18–21, 2019, Valencia, Spain

© 2019 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-6745-5/19/06.

<https://doi.org/10.1145/3319499.3328229>

regard to existing de facto standard UI frameworks like Angular which hinder their industry wide usage and adoption in practice. As mentioned in [5], there is no consensus on development methodologies for adaptive UIs due to a lack of standardization and because of largely varying goals from one interactive application to another. Beside that, after more than 30 years of research, we can observe that model-driven UI development is still not a common practice in industrial UI development [7]. The main reason for this is probably the steep learning curve of model-driven development approaches. Developers must first familiarize themselves with new and domain-specific languages, tools, concepts, and methods before they can apply such approaches.

In contrast, (web-)developers are mostly accustomed to very flexible concepts, as many modern development and UI frameworks follow a component-based UI development style that promotes the decomposition of UIs into individual components. While existing UI frameworks like Angular¹ or Vaadin² offer a variety of components for supporting the implementation of complex UIs, the provided components do not explicitly cover aspects like context monitoring and UI adaptation at widget level.

To address the aforementioned issues, we present a framework for component-based development of adaptive UIs. Inspired by the *Context of use Modulable widget (COMET)* framework [2], which was established as a theoretical framework, we implement a Component-Based Framework for Adaptive UIs (CoBAUI) based on Angular, as one of the currently most used framework for development of advanced web applications. In addition, to demonstrate the benefit of our approach, we implement and present a case study where we use our framework for the development of an application with specific adaptation requirements.

The rest of the paper is structured as follows: we subsequently sketch and elaborate our solution framework and present implementation details of the CoBAUI framework. After presenting a concrete case study, we discuss related work in terms of existing approaches for component-based development of adaptive UIs. Finally, we conclude and provide an outlook on future work.

2 THE COBAUI FRAMEWORK

In this section, we present our framework for component-based development of adaptive UIs - the CoBAUI framework. According to [9], “A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third party”. In Component-Based UI Development (CBUID), UIs

are composed of individual components, each component contributing to the overall interaction goal. CBUID aims to further reduce the complexity of UI development by leveraging reuse and modularization. Portions of the user interface and their related functionalities are encapsulated by components and can only be accessed via well-defined interfaces that hide implementation details to the outside. By constructing components first and then assembling the user interface, a bottom-up development approach is achieved where smaller components are used and combined to create more complex components and finally the whole UI.

Our CoBAUI framework follows the principle of separation of concerns in order to decouple the responsibilities of an adaptive UI. Based on our previous works regarding context management [10] and UI adaptation [12], we have identified three crucial aspects in respect to the development of adaptive UIs: *Context Capturing*, *Decision Making*, and *Adaptation Logic*. Figure 1 depicts the proposed component-based architecture for an adaptive UI. Each area of concern in an adaptive UI is covered by a specific component type.

Each component serves a well defined purpose and it fulfills it in an isolated and decoupled manner, following the single responsibility principle. In order to address this challenge, we have identified four types of components that deal with the concerns of an adaptive UI: *Context Provider*, *Rule Provider*, *Rule Evaluator*, and *Adaptive Widget*. A central component that serves as a connector is the *Adaptation Controller*. It enables communication and collaboration of all components as a coherent UI. In the following, we describe the different aspects of adaptive UIs.

Context Capturing

To manage the acquisition of the context-of-use in a more systematic and reusable way, *Context Provider* components encapsulate the logic for monitoring and processing contextual information. Individual context information is provided from various *SensorAPI* components for further processing. Encapsulating and modularizing the context information offers a flexible and comfortable solution approach, since it provides freedom on the implementation side and at the same time guarantees compatibility with the adaptive UI by using a uniform interface.

Decision Making

In order to automatically detect the need for UI adaptation at runtime, there must be rules that determine under which contextual conditions, adaptation actions should be performed. Rules give control over the adaptive behavior allowing an adaptation strategy for the UI to be defined in a straightforward and comprehensive way. Rules have to be evaluated at runtime on the basis of latest context information. They

¹<https://angular.io/>

²<https://vaadin.com/>

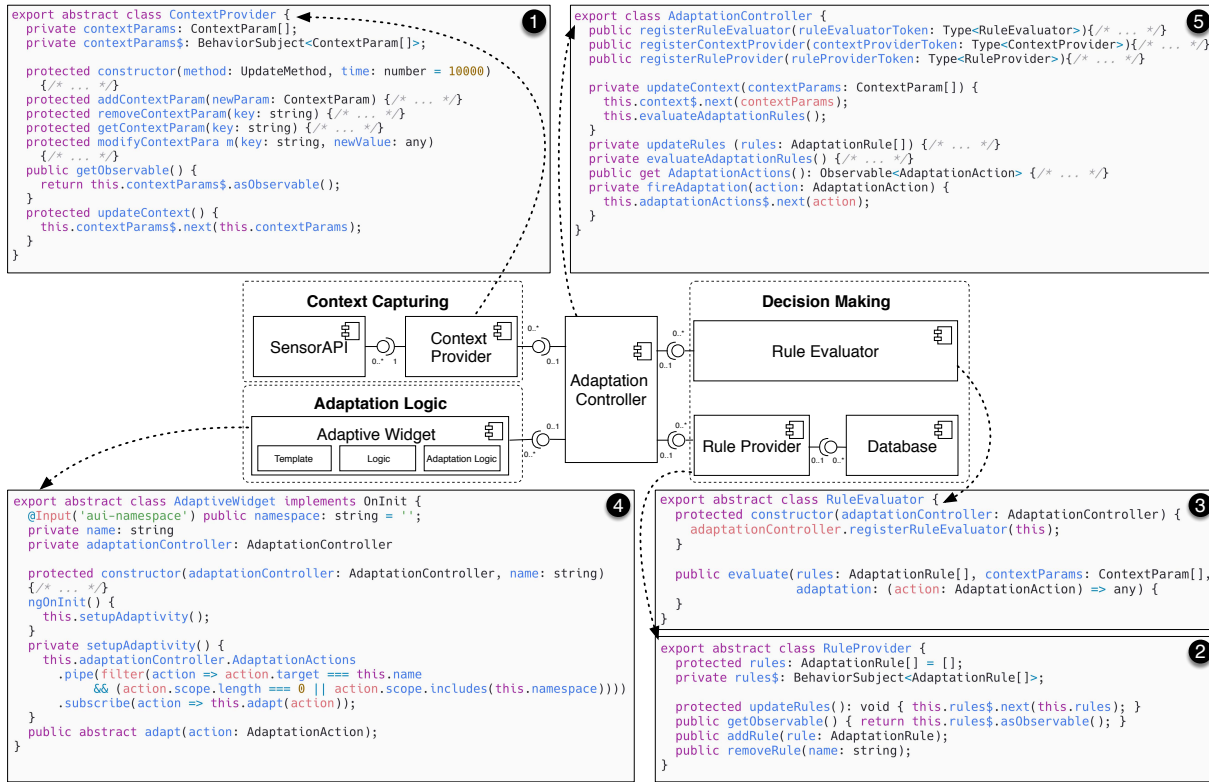


Figure 1: Component-Based Architecture of an Adaptive UI

should be specified in a universal and portable representation which is loaded dynamically from different sources at runtime.

For provisioning adaptation rules at runtime the *Rule Provider* component communicates with a *Database*. The *Rule Provider* component type enables a high degree of flexibility and extensibility. In addition, it allows incorporating dynamic changes to its set of rules. The *Rule Evaluator* component ensures evaluating rules at runtime and performing the UI adaptations. This component allows encapsulation in respect to the rule evaluation as well as decoupling which ensures technology independence.

Adaptation Logic

UI adaptation is the ability of a UI to react to a change in the context-of-use. There are different types of adaptations that can affect the layout, style, content or navigation of the UI. Widgets are UI components that encapsulate a portion of the UI. They are usually rather simple and serve as reusable building blocks to create more complex widgets by hierarchical composition. Besides a template that describes its visual appearance, a widget usually contains also a functional specification. We establish the *Adaptive Widget* component, which extends a widget with adaptation capabilities. In addition to its visual representation - *Template* and basic functionality - *Logic*, an *Adaptive Widget* also defines its own adaptation

functionality - *Adaptation Logic*. Each *Adaptive Widget* offers a specific adaptation feature of the UI. Therefore, the set of *Adaptive Widget* components depends on the adaptation feature that should be provided. The *Adaptation Controller* is a core component in the CoBAUI framework and an inherent part of the proposed adaptive UI. Its purpose is to connect the individual components in an application, to form a cohesive adaptive UI and to enable interaction between them. It receives context information from the *Context Provider* and adaptation rules from the *Rule Provider* component and it bundles them to form the applications overall context profile. A *Rule Evaluator* component can use the *Adaptation Controller* as its source of truth to obtain the current contextual information and adaptation rules for further evaluation. In return, the *Adaptation Controller* receives adaptation events triggered by the *Rule Evaluator* and forwards them to the targeted *Adaptive Widgets*.

3 IMPLEMENTATION

In this section, we elaborate on the implementation of the CoBAUI framework³, as a modular framework for development of adaptive UIs. Based on the Angular framework, we implemented the previously described concepts which resulted in an Angular library of UI components and services

³<https://github.com/Ferhat67/CoBAUI>

for supporting the development of adaptive UIs. While implementing the various components of the CoBAUI framework, special attention was paid to avoid unnecessary dependencies. The objective that we follow is that components can be exchanged and added with very little effort and intervention in other applications and projects. To reduce the coupling between the components, strong use is made of the Observer Pattern and Reactive Programming using RxJS⁴. In this way, components exchange data with a dynamic number of other components without maintaining explicit references. Furthermore, the Dependency Injection pattern offered by Angular is utilized. This is of great help as dependencies to other components can be resolved automatically at runtime.

In the following, we introduce the base classes for the different component types in which shared functionalities are implemented and which can be used by inheritance and extension for development of concrete components.

Context Provider

An excerpt from the *ContextProvider* base class ❶ is illustrated in Figure 1. As we can see, it maintains an array of the latest context parameters it has acquired. Furthermore, it supports two methods for notifying observers about updated context information - either event-triggered or at fixed time intervals. During initialization, the constructor allows passing options to specify its update method and it registers at the *Adaptation Controller*. An observable subject *contextParams* is used to publish updated context information to subscribed observers. Furthermore, the base class offers a number of helper methods to manage and modify context parameters. To report changes to observers, the *updateContext* method is provided, which is automatically triggered in case of the interval update method. Context information received from a Context Provider component is held in the form of *ContextParam*. The value of the context information is associated with a key, that is used to identify the context information. Additional meta information such as the context-of-use classification and value type can also be specified. As Typescript allows interfaces to be extended by inheritance, additional information for *ContextParam* can easily be incorporated when needed.

Rule Provider

A *RuleProvider* ❷ component has a comparable design to a *ContextProvider*, as it holds a set of *AdaptationRules* and an observable subject of rules. On initialization it injects and registers itself within the *Adaptation Controller*. If the set of rules is modified, it publishes the new set of rules via an appropriate *updateRules* method. The *AdaptationRule* interface specifies the structure of an adaptation rule. It consists

of a condition, that can be used to describe the context-of-use for which it applies and defines actions to specify the adaptation action events that will be performed. Additionally each rule has a name for identification purposes. The interface can be extended by additional properties and signatures if needed.

Rule Evaluator

A *RuleEvaluator* ❸ component is designed to be used by the *AdaptationController*. Thus, on initialization, this component registers itself to the *Adaptation Controller*. As depicted in Figure 1, the *evaluate* method provides evaluation of a concrete rule, based on current contextual information and triggering a specific *AdaptationAction* for the adaptive UI.

Adaptive Widget

As illustrated in Figure 1, the *AdaptiveWidget* ❹ base class implements the basic structure and functionality of an Adaptive Widget component. In the constructor, the *Adaptation Controller* component is requested to be injected by Angular's dependency injection mechanism. The constructor also requires a name to identify the widget component. By calling the *setupAdaptivity* method, the Adaptive Widget registers to receive *AdaptationAction* events from the *AdaptationController* component. The *adapt* method provides the adaptation logic for each inheriting widget.

Adaptation Controller

Every component has to register with the *AdaptationController* ❺ component in order to be a member of a coherent adaptive UI. For this reason, this core component, offers corresponding methods for which other components can register. The *AdaptationController* merges context parameters to a single set that represent the context-of-use of the application and it also combines all rules into a single set. As soon as changes occur to this information via the *updateContext* or *updateRules* methods, a re-evaluation of the rules starts based on the context information. *AdaptationAction* events received by the *RuleEvaluator* during the evaluation process are forwarded by the *adaptationAction* observable via the *fireAdaptation* method. By extending these base classes and interfaces, the development of concrete adaptation features for UIs is supported.

4 CASE STUDY - LIBRARY WEB APPLICATION

This section shows the benefit and usefulness of our CoBAUI framework based on a case study - *LibSoft*⁵. *LibSoft* is a web-based software for management of a university library. It allows students to browse the library's book inventory,

⁴<https://angular.io/guide/rx-library>

⁵<https://github.com/Ferhat67/LibSoft>

reserve books, and check the loan periods. Library staff can also use the application to browse the inventory and obtain a view of the reserved and issued books.

In the following, we present exemplary Adaptation Requirements (ARs) on how the UI should adapt in order to support specific context-of-use scenarios. Based on these ARs, we present the usage of our CoBAUI framework for developing adaptive UIs. The ARs include following scenarios: When using their smartphone, users must be able to easily access the navigation menu located in the sidebar. Depending on the user's handedness, the toggle button for the sidebar should be placed on the left or right side of the header to be operated with one hand (AR1). The LibSoft application can be used at any time of the day. To be easier to read in darker environments and to minimize eye strain, darker background colors and brighter font colors should be used (AR2). New users should receive additional help to become more acquainted with the LibSoft application. The menu items on the sidebar should describe their purpose in more detail. Experienced users, on the other hand, should not be bothered by help texts and should see a more compact navigation menu containing only icons (AR3). Figure 2 illustrates the architecture of the LibSoft application, based on the CoBAUI framework.

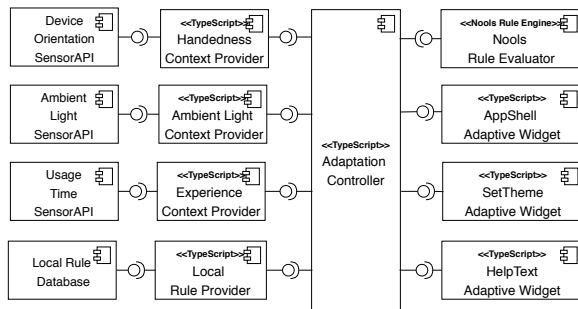


Figure 2: Case Study: Implementation of Adaptive UI for LibSoft by using the CoBAUI Framework

Context Capturing in LibSoft Application

In order to fulfill the adaptation requirements for LibSoft, the application must monitor various context information. For this purpose, *Context Provider* components are defined that manage the monitoring and gathering of data from different sources. Information about the user's handedness is required in order to fulfill AR1. Information about the brightness of the environment, in which the application is used, is needed to realize AR2. As an exemplary indicator for AR3, the time the user has worked with the application is required. To fulfill the adaptation requirements for our case study, we have implemented three different *Context Provider* components responsible for provisioning the contextual parameters. As

the required context parameters are *Handedness*, *Brightness*, and *Experience*, we have implemented the following *Context Provider* components: *Handedness Context Provider*, *Ambient Light Context Provider* and *Experience Context Provider*. The *Handedness Context Provider* component uses the gyroscope sensor of the mobile device to detect the inclination of the device. To access the gyroscope sensor, it uses the DeviceOrientationSensorAPI⁶, which is supported by the Google Chrome browser. In order to obtain information about the brightness of the environment, the *Ambient Light Context Provider* component uses the AmbientLightSensorAPI⁷ to access the device's ambient light sensor. The API is currently supported by the Google Chrome browser for Android. To judge the user experience, the *Experience Context Provider* component measures the amount of time in minutes the user has worked with the application. This information is stored persistently in the browser via the UsageTimeSensorAPI⁸ in order to be resumed with the next use of the application.

Decision Making in LibSoft Application

Since the adaptive runtime behavior of the widgets is to be controlled by specifying rules, a *Rule Provider* component is needed to obtain and provide rules. Furthermore, a *Rule Evaluator* component is necessary to evaluate rules at runtime and to trigger the adaptation events. The adaptation rules for this case study are located in a dedicated folder within the application - (in Figure 2 denoted as *Local Rule Database*). The rules are specified in JSON format. An excerpt of the Adaptation Rules for LibSoft Application is presented in Figure 3. Rules are retrieved by HTTP requests and for this purpose, a *Local Rule Provider* component is implemented that allows to load rules from a specified URL at runtime. A rule must be evaluated at runtime by a *Rule Evaluator* component which is responsible to check for each rule, whether its condition applies to the latest context information. In that case, it triggers adaptation events that are specified in the rule. To implement the evaluation process, we used the Nools⁹ rule engine within the *Nools Rule Evaluator* component.

```
1 [{
2   "name": "LEFTHANDED",
3   "condition": "c.user.handedness == 'left'",
4   "actions": [
5     { "name": "SET_MENU_POSITION", "params": { "position": "left" },
6       "target": "AppShellAW", "scope": [] } },
7   {
8     "name": "RIGHTHANDED",
9     "condition": "c.user.handedness == 'right'",
10    "actions": [
11      { "name": "SET_MENU_POSITION", "params": { "position": "right" },
12        "target": "AppShellAW", "scope": [] } }
13  ] }
```

Figure 3: Excerpt of the Adaptation Rules for LibSoft App

⁶<https://w3c.github.io/deviceorientation>

⁷<https://www.w3.org/TR/ambient-light>

⁸<https://www.w3.org/TR/webstorage/>

⁹<http://noolsjs.com/>

Adaptation Logic in LibSoft Application

For enabling the *Adaptation Logic*, we implemented several *Adaptive Widgets* based on the ARs determined beforehand. These *Adaptive Widgets* support the adaptive behaviour of the UI. The *Adaptive Widgets* in scope of the LibSoft application are the following:

AppShell Adaptive Widget. This component serves as a wrapper for the content of the LibSoft application and implements the logic for the header and the sidebar. In scope of this component, we implement the functionality regarding the navigation menu (AR1) which is located in the sidebar. Therefore, based on the handedness of the user, this component allows the navigation menu to be presented on the left or right side of the UI (Figure 4).

SetTheme Adaptive Widget. This component implements the adaptation logic for AR2 - adaptation due to changes in the brightness. Based on the ambient light brightness level, it allows enabling a light or dark colour theme for the UI (Figure 5).

HelpText Adaptive Widget. Based on the obtained contextual information about the usage time, this component allows that the menu items are shown as icons or display descriptive labels about their functions. The implementation of this component fulfills the requirement AR3 (Figure 6).

Adaptation Control in LibSoft Application

As every component type handles a separate concern in an isolated manner, an intermediary component is needed to establish a communication at runtime. For this purpose, we implement the *Adaptation Controller* component which is the centerpiece of the proposed architecture of an adaptive UI. Each of the previously mentioned components must register within the *Adaptation Controller*. After assembling the adaptive UI, the adaptive behavior of the UI is controlled and executed by defining adaptation rules.

5 RELATED WORK

Several approaches for component-based development of adaptive UIs have already been established in the scientific literature. Based on the following criteria, we compare existing approaches regarding component-based development of adaptive UIs: Support of Widget Level Adaptation (C1). Modular solution architecture which ensures separation of concerns, flexibility, and extensibility (C2). Dynamic rule-based control over UI adaptation (C3). Ease of access for a contemporary web developer based on a common Web Framework (C4).

The COMET ([2], [6]) framework has been introduced as an architectural style to support widget level adaptation of UIs. It is described as a framework which supports context-aware widget adaptation (C1). However, the architecture

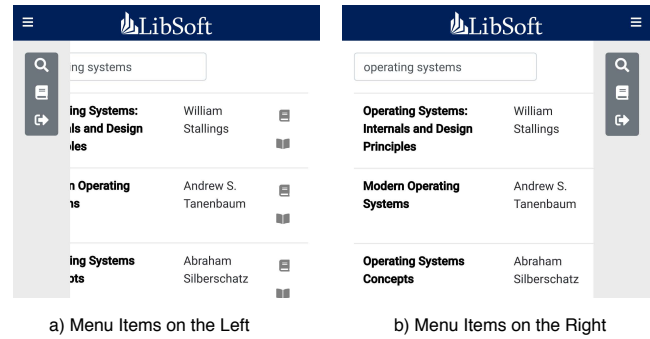


Figure 4: UI adaptation based on handedness

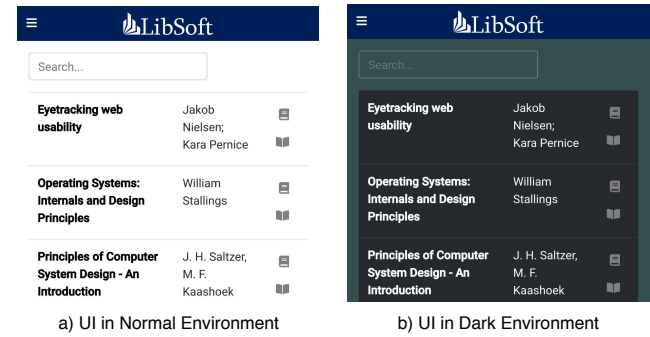


Figure 5: UI adaptation based on lighting conditions

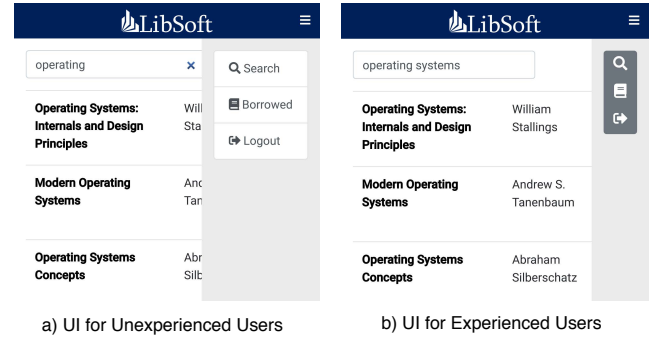


Figure 6: UI adaptation based on usage time

does not propose how context information should be obtained and provided (C2). It also does not entail a dynamic rule-based control of the adaptive behavior (C3) at runtime. Furthermore, the lack of an implementation of the COMET approach based on a common UI framework like Angular makes it difficult to use for contemporary web developers (C4).

Another approach which promotes the component-based development of adaptive UIs is CRUISe (Composition of Rich User Interface Services) [8]. It is a model-driven approach for dynamic composition of UIs which incorporates a service-oriented architecture at the presentation layer (C1). According to CRUISe, a UI is composed of components that are obtained from distributed repositories and integrated by the

client (C2). A component model defines the structure, interface and packaging of a UI component encapsulated by a User Interface Service (UIS). However, the composition description is static and no explicit adaptation actions can be chosen or controlled in respect to this framework (C3). In respect to ease of access, there is no practical implementation of CRUISe (C4).

In terms of web development, *Responsive Web Design (RWD)* is a widely practiced approach for adapting web pages to different devices and device properties. RWD has contributed significantly to the popularity of CSS (Cascading Style Sheets) component libraries. However, components denote reusable style templates for common web interface components such as buttons, lists, and forms. Unfortunately, the adaptation features are limited to the visual properties of the UI and it offers no possibility to access information about the user or the environment and not all context-of-use categories are supported (C1). RWD is not modular in terms of concerns, as the browser handles both context monitoring and rule evaluation and cannot be extended (C2). However, rules are evaluated by the browser at runtime and they can be dynamically changed (C3). Also, RWD is suitable for use with standard web technologies and UI frameworks like Angular (C4).

	Criteria			
	Widget Level Adaptation	Modular Architecture	Dynamic UI Adaptation Control	Ease Of Access
COMET	●	◐	○	○
CRUISe	◐	●	○	○
Responsive Web Design	◐	○	●	●
Web Components	●	○	○	●
COBAUI Framework	●	●	●	●

Figure 7: Comparison of Related Work

Another component-based approach for adaptive UIs is *Web Components*¹⁰. It is a W3C specification, that enables web developers to define custom HTML elements with their own tags and encapsulate their own template to be rendered on the screen. Web Components enable component-based development for UIs and encapsulate their own functionality. They can be used to construct adaptable widgets and due to their portability, components can be bundled in libraries and deployed in multiple applications (C1). Unfortunately, Web Components do not provide a holistic solution to implement adaptive UIs (C2). This approach do not offer the ability to monitor context information and to determine adaptive behavior (C3). Since Angular uses its own implementation of Web Components, (C4) is also fulfilled. As shown in Figure

7, compared to all these approaches, our CoBAUI framework fulfills all of the established criteria.

6 CONCLUSION AND OUTLOOK

In this paper, we present the CoBAUI framework as a modular architecture which promotes component-based development of adaptive UIs. Our framework supports implementation of adaptive UIs in a decoupled and isolated manner. The main contribution of this work is an implementation of the COMET framework [2], which is introduced on a conceptual level. The benefit and usefulness of our framework is shown by implementing a concrete case study - LibSoft application. Our approach can serve as a basis for further investigations due to its extensibility. Efficiency and effectiveness of the CoBAUI framework should also be assessed in relation to other AUI development approaches. Although this paper deals exclusively with adaptations of graphical user interfaces (GUIs), further improvements could be included to support multi-modal UI adaptations.

REFERENCES

- [1] Pierre A. Akiki, Arosha K. Bandara, and Yijun Yu. 2013. RBUIS: simplifying enterprise application user interfaces through engineering role-based adaptive behavior. In *EICS'13*.
- [2] Gaëlle Calvary, Joëlle Coutaz, Olfa Dâassi, Lionel Balme, and Alexandre Demeure. 2004. Towards a New Generation of Widgets for Supporting Software Plasticity: The "Comet". In *EHCI-DSVIS'04*.
- [3] Gaëlle Calvary, Joëlle Coutaz, and David Thevenin. 2001. A Unifying Reference Framework for the Development of Plastic User Interfaces. In *EHCI'01*.
- [4] Gaëlle Calvary, Joëlle Coutaz, David Thevenin, Quentin Limbourg, Laurent Bouillon, and Jean Vanderdonckt. 2003. A Unifying Reference Framework for multi-target user interfaces. *Interacting with Computers* (2003).
- [5] Benoît Collignon, Jean Vanderdonckt, and Gaëlle Calvary. 2008. Model-Driven Engineering of Multi-target Plastic User Interfaces. In *ICAS'08*.
- [6] Alexandre Demeure, Gaëlle Calvary, and Karin Coninx. 2008. COMET(s), A Software Architecture Style and an Interactors Toolkit for Plastic User Interfaces. In *DSV-IS 2008*.
- [7] Gerrit Meixner, Fabio Paternò, and Jean Vanderdonckt. 2011. Past, Present, and Future of Model-Based User Interface Development. *i-com* (2011).
- [8] Stefan Pietschmann, Martin Voigt, Andreas Rümpel, and Klaus Meißner. 2009. CRUISe: Composition of Rich User Interface Services. In *ICWE'09*.
- [9] Clemens A. Szyperski, Dominik Gruntz, and Stephan Murer. 2002. *Component software - beyond object-oriented programming, 2nd Edition*. Addison-Wesley.
- [10] Enes Yigitbas, Silas Grün, Stefan Sauer, and Gregor Engels. 2017. Model-Driven Context Management for Self-adaptive User Interfaces. In *UCAmI 2017*.
- [11] Enes Yigitbas, Stefan Sauer, and Gregor Engels. 2017. Adapt-UI: an IDE supporting model-driven development of self-adaptive UIs. In *EICS'17*.
- [12] Enes Yigitbas, Hagen Stahl, Stefan Sauer, and Gregor Engels. 2017. Self-adaptive UIs: Integrated Model-Driven Development of UIs and Their Adaptations. In *ECMFA 2017*.

¹⁰<https://www.webcomponents.org>